B3 - Qualité de Code (QDC)

RNCP 37873

FAAS Guillaume

Developer Advocate, Software Craftsman, Speaker, Coach, Mentor

GitHub /Tr00d

/les-tontons-crafters

/guillaumefaas

SOFTWARE CRAFT LUXEMBOURG

DEV /Tr00d

sessionize /guillaume-faas

Déroulement du cours

Objectifs du cours

- Comprendre l’état d’esprit du Software Craftsmanship, ses origines, ses valeurs et son lien avec l’Agilité.
- Mettre en oeuvre les principes de Clean Code pour améliorer la lisibilité et la maintenabilité du code.
- TDD afin de piloter son implémentation par les tests.
- Itérer sur du code “legacy” de manière sécurisée, en identifiant les code smells et en s’appuyant sur des tests.
- Identifier les liens entre qualité du code et éco-conception en montrant comment certaines pratiques permettent de limiter la dette technique et l’empreinte environnementale du logiciel.
- Appréhender les tests de montée en charge pour explorer de nouvelles façons de tester.

Formats que l’on appliquera cette semaine

- Présentations
- Échanges collaboratifs
- Ateliers techniques **en Pair Programming**
- Feedbacks collaboratifs

Prérequis techniques

Mature

Typage fort

Multi-paradigme

Facile à lire/comprendre

.NET en support

Gestion de package simple (NuGet)


.NET
Cloud
Web
Desktop
Mobile
Gaming
IoT
AI
Visual Studio
Visual Studio Code
CLI
Tools
+
Windows
Linux
macOS
Operating system
+
NuGet
GitHub
Components, tools, library vendors
Ecosystem

Prérequis

- SDK 9.0: https://dotnet.microsoft.com/en-us/download/dotnet/9.0

- IDE de votre choix:
  - Jetbrains Rider: https://www.jetbrains.com/rider/
  - Visual Studio Community (Windows, MacOS) : https://visualstudio.microsoft.com/fr/vs/community/
  - Visual Studio Code: https://code.visualstudio.com/ && C# Dev Kit: https://marketplace.visualstudio.com/items?itemName=ms-dotnettools.csdevkit

La “qualité”… C’est quoi?

```
public static bool v(string t)
{
    if (t == null)
        return false;
    if (t.Length < 8 || t.Length > 15)
        return false;
    if (!t.StartsWith("+"))
        return false;
    for (int i = 1; i < t.Length; i++)
    {
        if (!int.TryParse(t[i].ToString(), out _))
        {
            return false;
        }
    }
    return true;
}
```

```
public class Account
{
    private const decimal DailyLimit = ...;
    private const decimal WeeklyLimit = ...;
    private decimal Balance { get; }
    private decimal OverdraftLimit { get; }
    private decimal DailyWithdrawn { get; }
    private decimal WeeklyWithdrawn { get; }

    public bool CanWithdraw(Money money) =>
        HasSufficientFunds(money.Amount)
        && IsWithinDailyLimit(money.Amount)
        && IsWithinWeeklyLimit(money.Amount)
        && IsTargetCurrencyAllowed(money.Currency);

    private bool HasSufficientFunds(decimal amount) => ...
    private bool IsTargetCurrencyAllowed(Currency targetCurrency) => ...
    private bool IsWithinDailyLimit(decimal amount) => ...
    private bool IsWithinWeeklyLimit(decimal amount) => ...

    public record Money(decimal Amount, Currency Currency);
}
```

```
public static bool v(string t)
{
    if (t == null)
    }
    return true;
}
```

PhoneNumberVerificationTest *(13 tests)* Success

- ShouldReturnFalse_GivenDoesNotStartWithPlusSign Success
- ShouldReturnFalse_GivenLengthIsHigherThan15 *(1 test)* Success
- ShouldReturnFalse_GivenLengthIsLowerThan8 *(7 tests)* Success
- ShouldReturnFalse_GivenNotAllCharactersAreNumerics Success
- ShouldReturnFalse_GivenValueIsEmpty Success
- ShouldReturnFalse_GivenValueIsNull Success
- ShouldReturnTrue_GivenValueIsValidE164 Success

Comment mesurer la qualité de code?

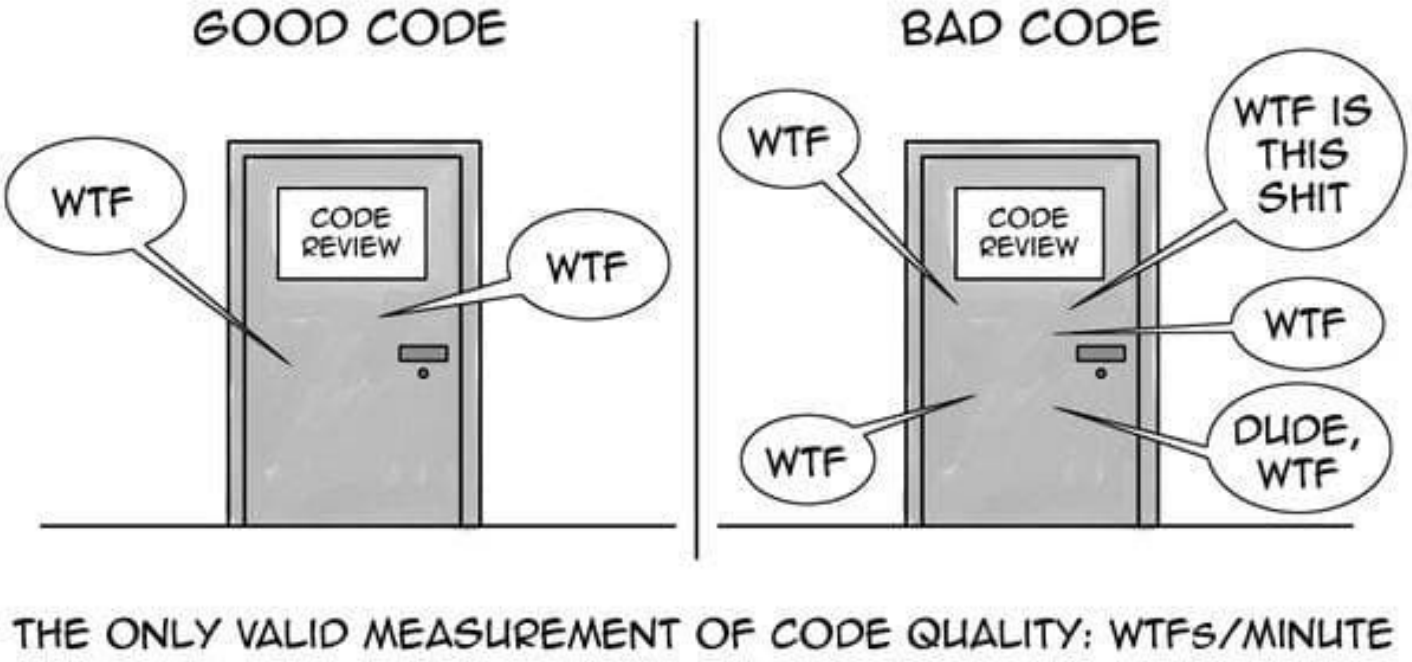
```
public static bool v(string t)
{
    if (t == null)
        return false;
    if (t.Length < 8 || t.Length > 15)
        return false;
    if (!t.StartsWith("+"))
        return false;
    for (int i = 1; i < t.Length; i++)
    {
        if (!int.TryParse(t[i].ToString(), out _))
        {
            return false;
        }
    }
    return true;
}
```

```
public class Account
{
    private const decimal DailyLimit = ...;
    private const decimal WeeklyLimit = ...;
    private decimal Balance { get; }
    private decimal OverdraftLimit { get; }
    private decimal DailyWithdrawn { get; }
    private decimal WeeklyWithdrawn { get; }

    public bool CanWithdraw(Money money) =>
        HasSufficientFunds(money.Amount)
        && IsWithinDailyLimit(money.Amount)
        && IsWithinWeeklyLimit(money.Amount)
        && IsTargetCurrencyAllowed(money.Currency);

    private bool HasSufficientFunds(decimal amount) => ...
    private bool IsTargetCurrencyAllowed(Currency targetCurrency) => ...
    private bool IsWithinDailyLimit(decimal amount) => ...
    private bool IsWithinWeeklyLimit(decimal amount) => ...

    public record Money(decimal Amount, Currency Currency);
}
```

10/10?

42?

0/10?



Source: Uncle Bob - “Clean Code: A handbook of Agile Software Craftsmanship”

Comment mesurer la qualité de code?

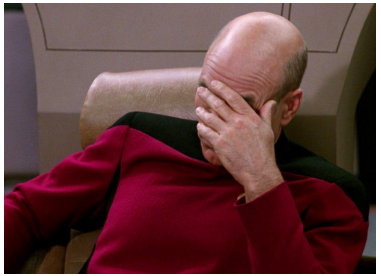
https://sonarcloud.io/

https://codescene.io/

```
5           public static bool v(string t)
6           {
7               if (t == null)
8                   return false;
9               if (t.Length < 8 || t.Length > 15)
10                  return false;
11              if (!t.StartsWith("+"))
```

Utilisez 'string.StartsWith(char)' à la place de 'string.StartsWith(string)' quand vous avez une chaîne avec un seul caractère

```
12                  return false;
13              for (var i = 1; i < t.Length; i++)
14                  if (!int.TryParse(t[i].ToString(), out _))
15                      return false;
16
17              return true;
18          }
```



Virtual Code Review

CodeQuality/.../PhoneNumberVerification.cs

Rule Violations

Warnings

Positive Trends

Pourquoi c’est si difficile à mesurer?

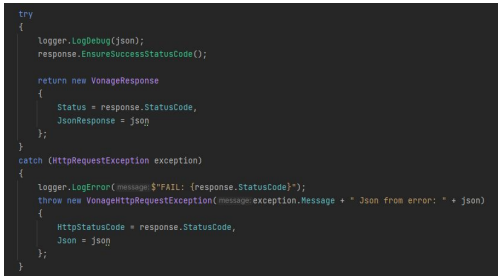
- Dépend du contexte et des priorités
- Les outils mesurent des indicateurs **techniques** et **locaux**
- Dimension humaine
- Ancré dans le temps

Qualité = un code facile à lire, comprendre et modifier

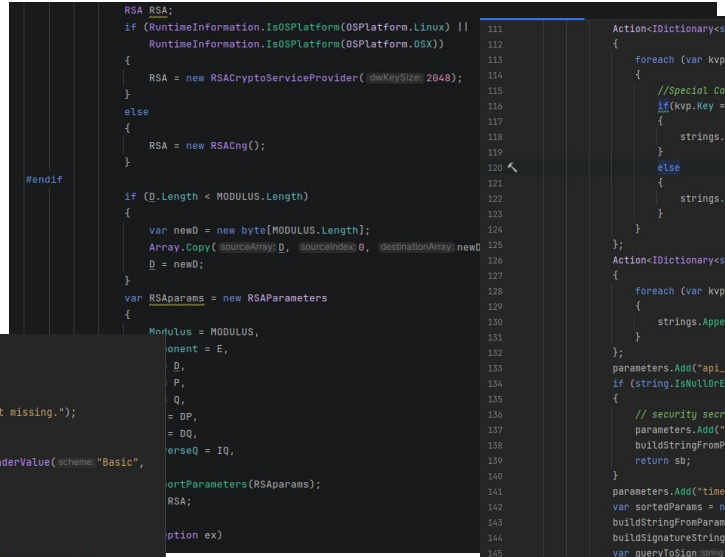
** Sans friction ni douleur*

La “qualité”... Pourquoi?

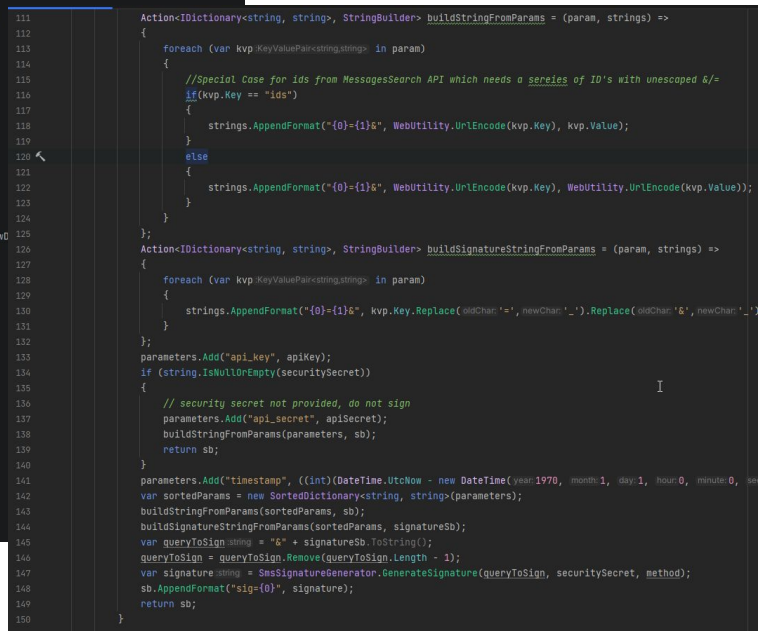
Le chaos comme toile de fond



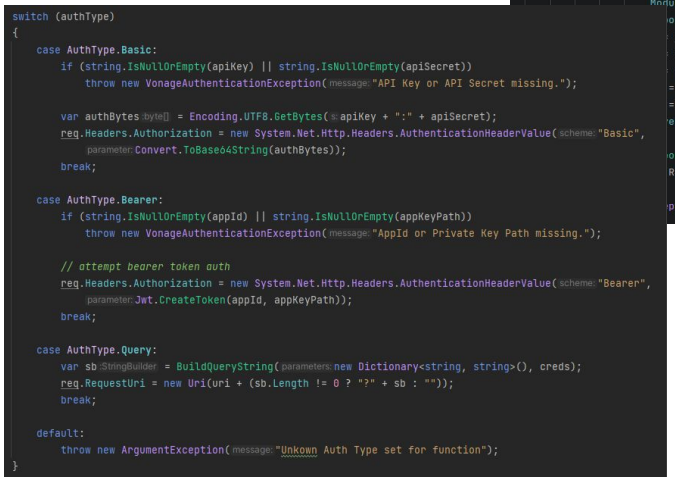
```
try
{
    logger.LogDebug(json);
    response.EnsureSuccessStatusCode();

    return new VonageResponse
    {
        Status = response.StatusCode,
        JsonResponse = json
    };
}
catch (HttpRequestException exception)
{
    logger.LogError(message:$"FAIL: {response.StatusCode}");
    throw new VonageHttpRequestException(message:exception.Message + " Json from error: " + json)
    {
        HttpStatusCode = response.StatusCode,
        Json = json
    };
}
```



```
        RSA RSA;
        if (RuntimeInformation.IsOSPlatform(OSPlatform.Linux) ||
            RuntimeInformation.IsOSPlatform(OSPlatform.OSX))
        {
            RSA = new RSACryptoServiceProvider( dwKeySize: 2048);
        }
        else
        {
            RSA = new RSACng();
        }
#endif

        if (D.Length < MODULUS.Length)
        {
            var newD = new byte[MODULUS.Length];
            Array.Copy( sourceArray: D,  sourceIndex: 0,  destinationArray: newD
            D = newD;
        }
        var RSAparams = new RSAParameters
        {
            Modulus = MODULUS,
            onent = E,
            D,
            P,
            Q,
            = DP,
            = DQ,
            erseQ = IQ,

        ortParameters(RSAparams);
        RSA;

        ption ex)
```

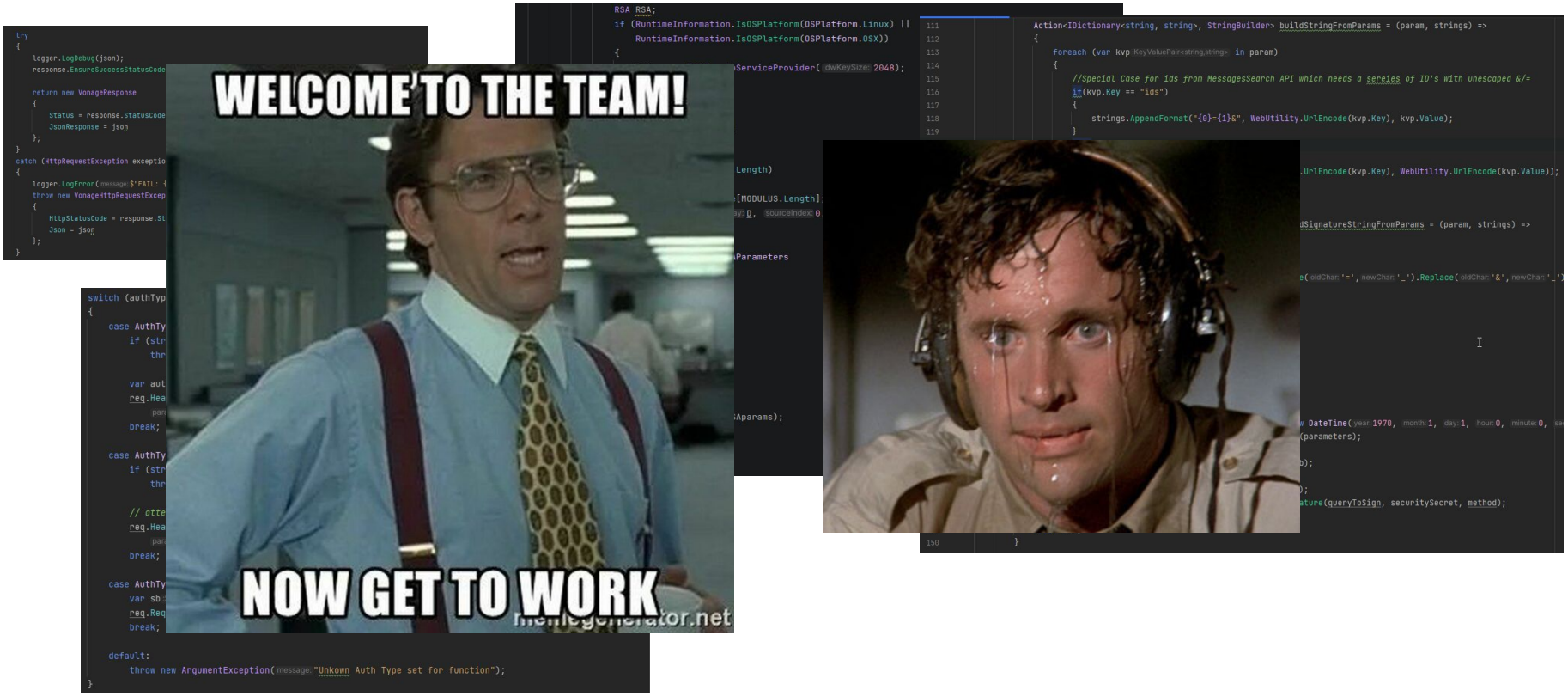


```
    Action<IDictionary<string, string>, StringBuilder> buildStringFromParams = (param, strings) =>
    {
        foreach (var kvp:KeyValuePair<string,string> in param)
        {
            //Special Case for ids from MessagesSearch API which needs a sereies of ID's with unescaped &/=
            if(kvp.Key == "ids")
            {
                strings.AppendFormat("{0}={1}&", WebUtility.UrlEncode(kvp.Key), kvp.Value);
            }
            else
            {
                strings.AppendFormat("{0}={1}&", WebUtility.UrlEncode(kvp.Key), WebUtility.UrlEncode(kvp.Value));
            }
        }
    };
    Action<IDictionary<string, string>, StringBuilder> buildSignatureStringFromParams = (param, strings) =>
    {
        foreach (var kvp:KeyValuePair<string,string> in param)
        {
            strings.AppendFormat("{0}={1}&", kvp.Key.Replace( oldChar: '=', newChar: '_').Replace( oldChar: '&', newChar: '_')
        }
    };
    parameters.Add("api_key", apiKey);
    if (string.IsNullOrEmpty(securitySecret))
    {
        // security secret not provided, do not sign
        parameters.Add("api_secret", apiSecret);
        buildStringFromParams(parameters, sb);
        return sb;
    }
    parameters.Add("timestamp", ((int)(DateTime.UtcNow - new DateTime( year: 1970,  month: 1,  day: 1,  hour: 0,  minute: 0,
    var sortedParams = new SortedDictionary<string, string>(parameters);
    buildStringFromParams(sortedParams, sb);
    buildSignatureStringFromParams(sortedParams, signatureSb);
    var queryToSign = signatureSb.ToString();
    queryToSign = queryToSign.Remove(queryToSign.Length - 1);
    var signature = SmsSignatureGenerator.GenerateSignature(queryToSign, securitySecret, method);
    sb.AppendFormat("sig={0}", signature);
    return sb;
}
```



```
switch (authType)
{
    case AuthType.Basic:
        if (string.IsNullOrEmpty(apiKey) || string.IsNullOrEmpty(apiSecret))
            throw new VonageAuthenticationException(message:"API Key or API Secret missing.");

        var authBytes:byte[] = Encoding.UTF8.GetBytes(s:apiKey + ":" + apiSecret);
        req.Headers.Authorization = new System.Net.Http.Headers.AuthenticationHeaderValue(scheme:"Basic",
            parameter:Convert.ToBase64String(authBytes));
        break;

    case AuthType.Bearer:
        if (string.IsNullOrEmpty(appId) || string.IsNullOrEmpty(appKeyPath))
            throw new VonageAuthenticationException(message:"AppId or Private Key Path missing.");

        // attempt bearer token auth
        req.Headers.Authorization = new System.Net.Http.Headers.AuthenticationHeaderValue(scheme:"Bearer",
            parameter:Jwt.CreateToken(appId, appKeyPath));
        break;

    case AuthType.Query:
        var sb:StringBuilder = BuildQueryString(parameters:new Dictionary<string, string>(), creds);
        req.RequestUri = new Uri(uri + (sb.Length != 0 ? "?" + sb : ""));
        break;

    default:
        throw new ArgumentException(message:"Unkown Auth Type set for function");
}
```

Le chaos comme toile de fond



La qualité répond à un problème **systémique** et **permanent**

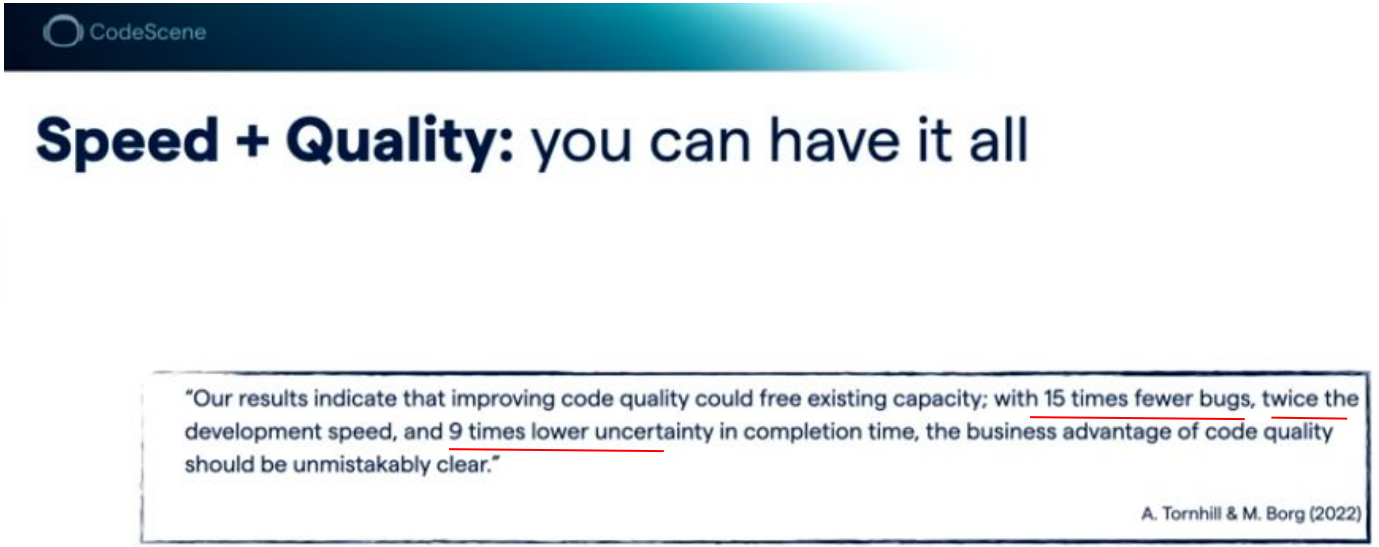
Lehman's Law of Software Evolution (1974):
"*The quality of a [...] system will appear to be declining unless it is rigorously maintained [...]*"
Source: https://en.wikipedia.org/wiki/Lehman%27s_laws_of_software_evolution

Mais... On a pas le temps!

“Project Management Triangle”

Mais... On a pas le temps!

La “vitesse” est une conséquence de la qualité



Source: Adam Tornhill “*Code Red - The business impact of code quality*”

https://www.youtube.com/watch?v=F8WuA5T1u7s

Qualité...
Pourquoi?

Pragmatisme **préventif**

Combattre l'**entropie logicielle**

Ecrire un code **compréhensible**

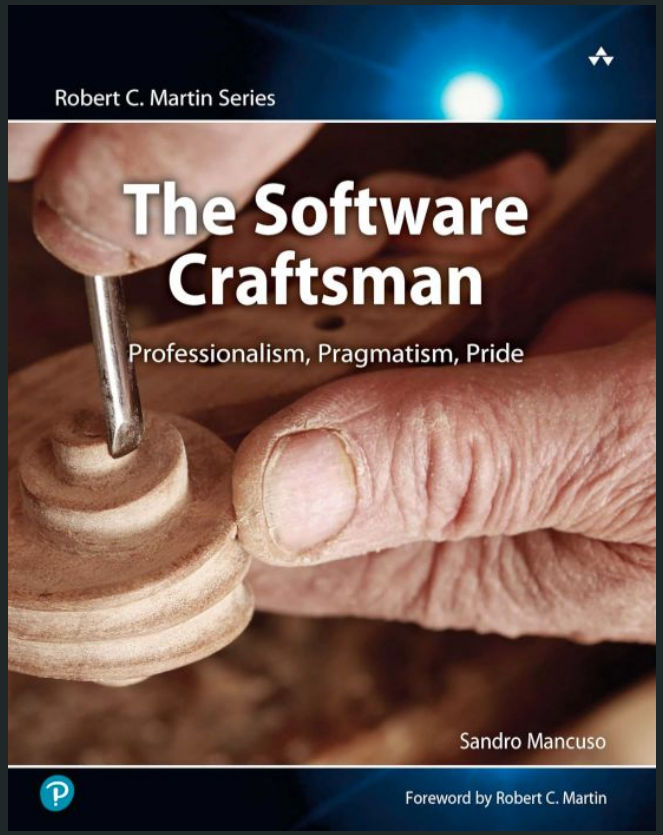
Le seul moyen d'aller **vite longtemps** (aka "**Sustainable Pace**")

Garder une codebase **sous contrôle**

La “qualité”… Comment?

Software Craftsmanship

Une réponse **culturelle** et **pratique** au besoin de qualité



Une extension de l’Agile Software Development

- Individuals and interactions over processes and tools
- Working software over comprehensive communication
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Agile Manifesto (2001): https://manifesto.softwarecraftsmanship.org/

Software Craftsmanship Manifesto (2009): https://manifesto.softwarecraftsmanship.org/

Une extension de l’Agile Software Development

- Not only individuals and interactions, but also **a community of professionals**
- Working software over comprehensive communication
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Agile Manifesto (2001): https://manifesto.softwarecraftsmanship.org/

Software Craftsmanship Manifesto (2009): https://manifesto.softwarecraftsmanship.org/

Une extension de l’Agile Software Development

- Not only individuals and interactions, but also **a community of professionals**
- Not only working software, but also **well-crafted software**
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Agile Manifesto (2001): https://manifesto.softwarecraftsmanship.org/

Software Craftsmanship Manifesto (2009): https://manifesto.softwarecraftsmanship.org/

Une extension de l’Agile Software Development

- Not only individuals and interactions, but also **a community of professionals**
- Not only working software, but also **well-crafted software**
- Not only customer collaboration, but also **productive partnerships**
- Responding to change over following a plan

Agile Manifesto (2001): https://manifesto.softwarecraftsmanship.org/

Software Craftsmanship Manifesto (2009): https://manifesto.softwarecraftsmanship.org/

Une extension de l’Agile Software Development

- Not only individuals and interactions, but also **a community of professionals**
- Not only working software, but also **well-crafted software**
- Not only customer collaboration, but also **productive partnerships**
- Not only responding to change, but also **steadily adding value**

Agile Manifesto (2001): https://manifesto.softwarecraftsmanship.org/

Software Craftsmanship Manifesto (2009): https://manifesto.softwarecraftsmanship.org/

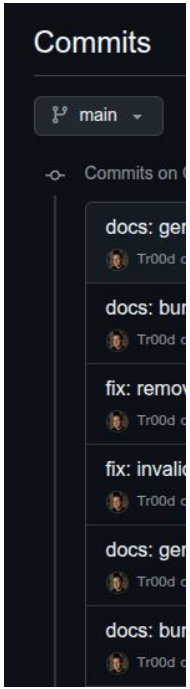
Agile = Build the **right** thing

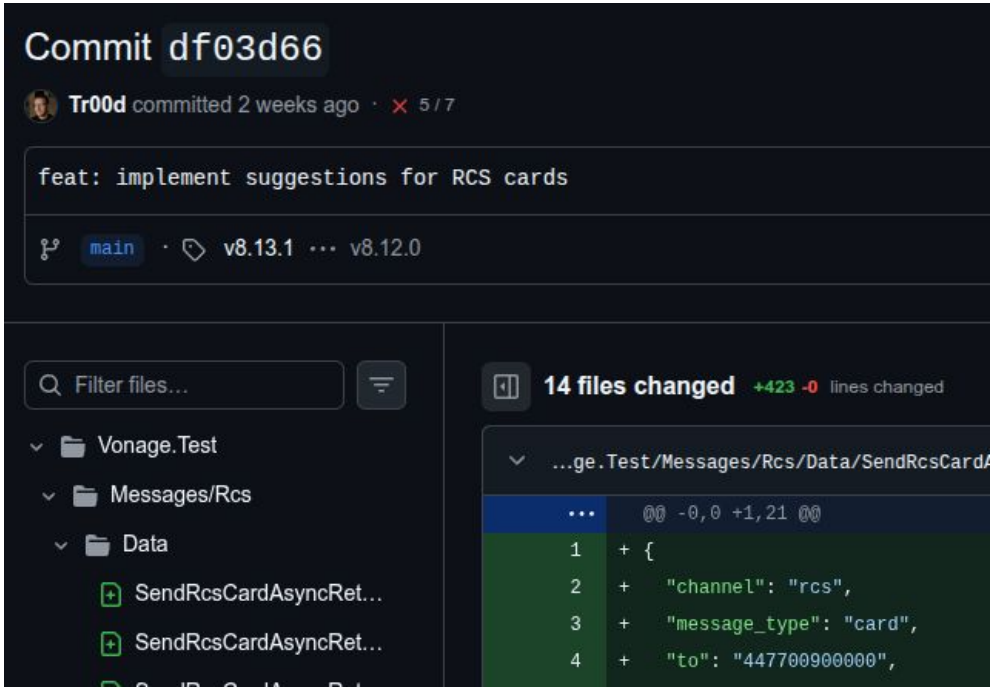
Craft = Build the thing **right**

Mindset “Craft”

Une posture professionnelle

Être fier de ce que l’on produit





Un standard de qualité

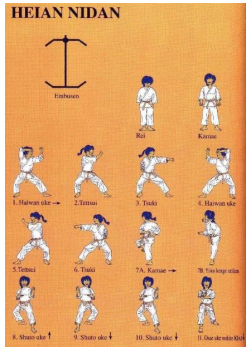
- Responsabilité vis-à-vis
  - Du code
  - De ses pairs (aka. autres développeurs)
  - Des utilisateurs et clients

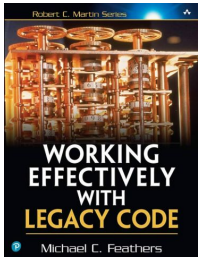
- Pragmatisme
  - Les bons gestes
  - Les bons outils
  - Les bonnes méthodologies

- Engagement à produire du code de qualité

Continuous Improvement

- Pratique / Katas
- Livres
- Meetups / User Groups
- Conférences





SOFTWARE CRAFT LUXEMBOURG

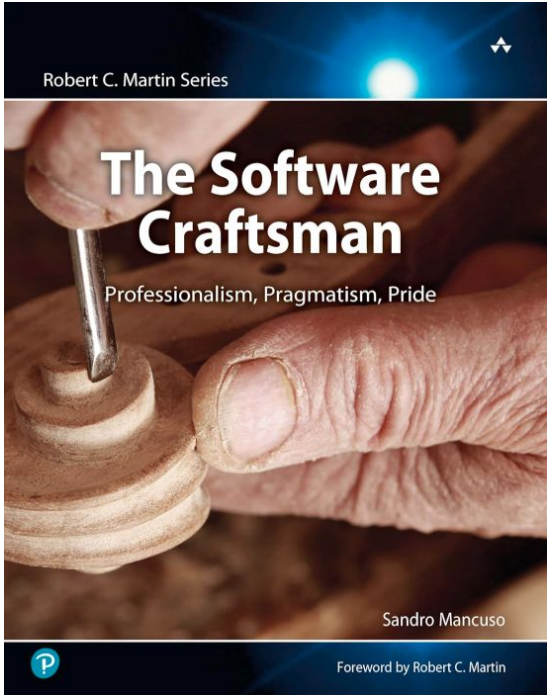
Partage de connaissance avec ses pairs

- Mentoring / Coaching
- Devenir speaker
- Animer/Organiser des évènements

Resources

- Egoless Crafting: https://egolesscrafting.org/

- Codurance's Fireside Chats: https://www.youtube.com/watch?v=Rrft6NappKQ&list=PLGS1QE37I5lTjbbOVm8QrS-85WAdeBehx

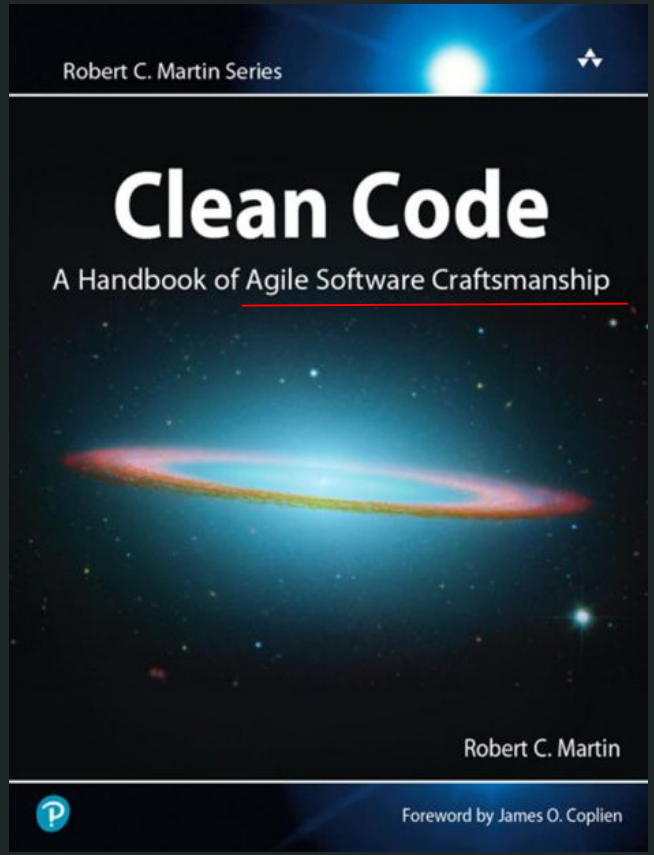


Retro time!

Clean Code

Clean Code

Apprendre à écrire du code **lisible**, **compréhensible**, et **facile à modifier**



“Code that works is not enough”

Les principes de *Clean Code*

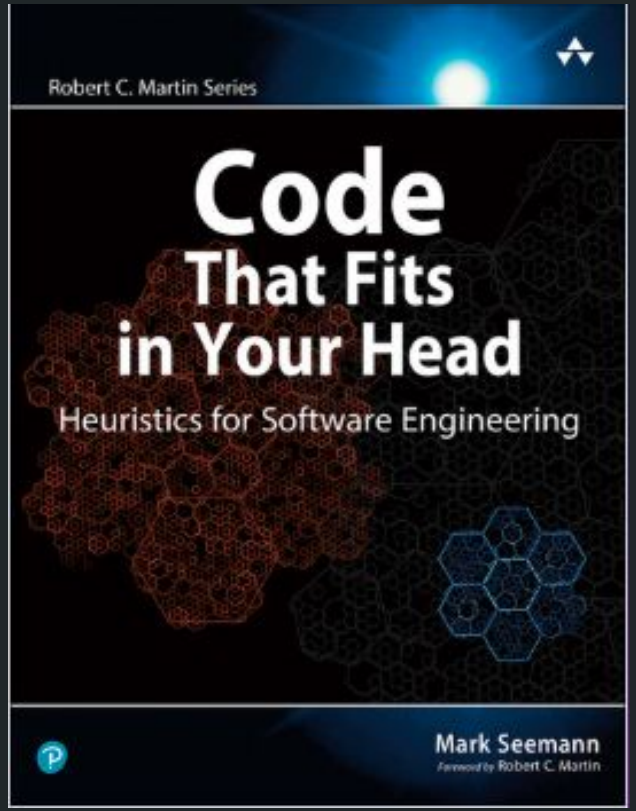
- **Lisibilité** - “Code is read far more often than it is written”

- **Simplicité** - Cohésion, implémentation minimale, éviter les optimisations prématurées

- **Expressivité** - Quoi > Pourquoi > Comment

Une affaire de charge mentale

Votre mémoire court-terme peut retenir entre 4 et 7 éléments



*** Pourquoi le Burger De La Mort du **Burger Quizz** comporte 10 questions? ***

You spend more time
reading *code than* ***writing*** *it*

“The ratio of time spent reading vs. writing is **well over 10:1**” -
Uncle Bob

“So… Optimize for ***readability****”*

Les pratiques de *Clean Code*

- Objectif n’est pas Clean, mais Clean**er**

- Quelques règles générales
  - **D**on’t **R**epeat **Y**ourself
  - **Y**ou **A**in’t **G**onna **N**eed **I**t
  - **K**eep **I**t **S**imple, **S**tupid
  - Principes **SOLID**
  - Boy Scout Rule

Les pratiques de *Clean Code* - **Naming**

*Objectif: Exprimer clairement une **intention***

- Utiliser des noms **explicites** et **cohérents**
- Priorité sur ce que fait votre élément (**What**), pas le comment (**How**)
- “*Ubiquitous Language*”

Les pratiques de *Clean Code* - **Fonctions**

Objectif: Des fonctions petites et lisibles

- Scope simple: complexité cyclomatique faible, peu de paramètres
- Eviter les paramètres de type `bool`
- “Command-Query Separation” (**CQS**)
- “Code Landscape”

Les pratiques de *Clean Code* - **Commentaires**

*Objectif: Documenter le **pourquoi** (au besoin), pas le **comment***

- Eviter les commentaires au maximum
- Un commentaire peut aider à expliquer **pourquoi**
- Les commentaires vieillissent mal

Les pratiques de *Clean Code* - **Objets et Responsabilités**

Objectif: Couplage, cohésion, et expliciter les responsabilités

- Principes **SOLID**
- Eviter les “*god class*”
- Principes **OOP** + “**Composition** over Inheritance”
- **Loi de Déméter**: “Don’t Talk to Strangers”

Les pratiques de *Clean Code* - **Tests unitaires**

Objectif: Confiance dans la suite de tests

- Chaque doit être **rapide**, **indépendant**, **répétable**, **lisible** et **déterministe**
- Une “assertion” (aka. **comportement**) par test
- Aucune logique conditionnelle dans un test
- Respecter la structure **A**rrange / **A**ct / **A**ssert

Les pratiques de *Clean Code* - **Refactoring**

Objectif: Améliorer sans casser

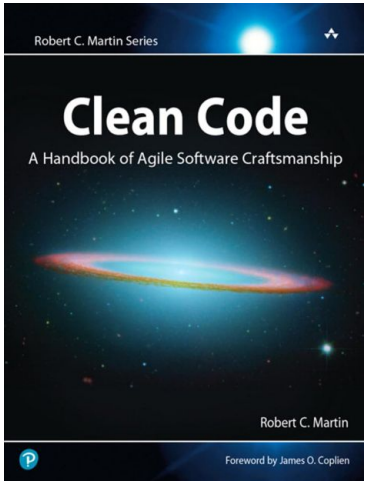
- Règle du Boy-Scout: “Always leave the campfire in a **better** state than you found it”
- Pratique **permanente** et **continue**
- Un code mort ou inutilisé n’a rien à faire dans la codebase
- **DRY** / **KISS** / **YAGNI**

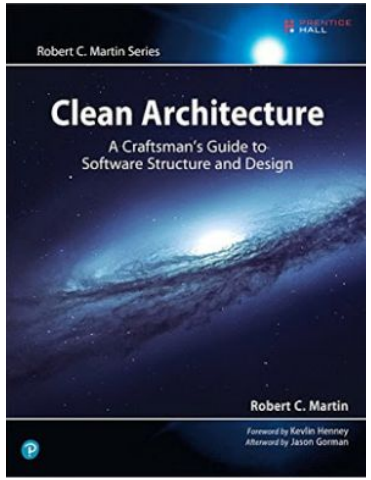
Object Calisthenics

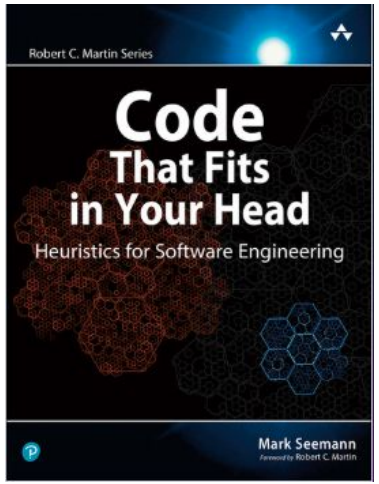
- One level of indentation per method
- Don’t use the “Else” keyword
- Wrap all primitives and Strings in classes
- First class collections
- One dot per line
- Don’t abbreviate
- Keep all classes less than 50 lines
- No classes with more than two instance variables
- No getters or setters

Resources

- Uncle Bob - “Clean Code”: https://www.youtube.com/watch?v=7EmboKQH8lM
- Uncle Bob - “SOLID Principles”: https://www.youtube.com/watch?v=zHiWqnTWsn4
- Uncle Bob - “The Clean Coder”: https://www.youtube.com/watch?v=NeXQEJNWO5w
- Mark Seemann - “Fractal Architecture”: https://www.youtube.com/watch?v=t3rSCpcJzm0
- Goat Review - “Object calisthenics & Clean Code”: https://goatreview.com/object-calisthenics-9-rules-clean-code/





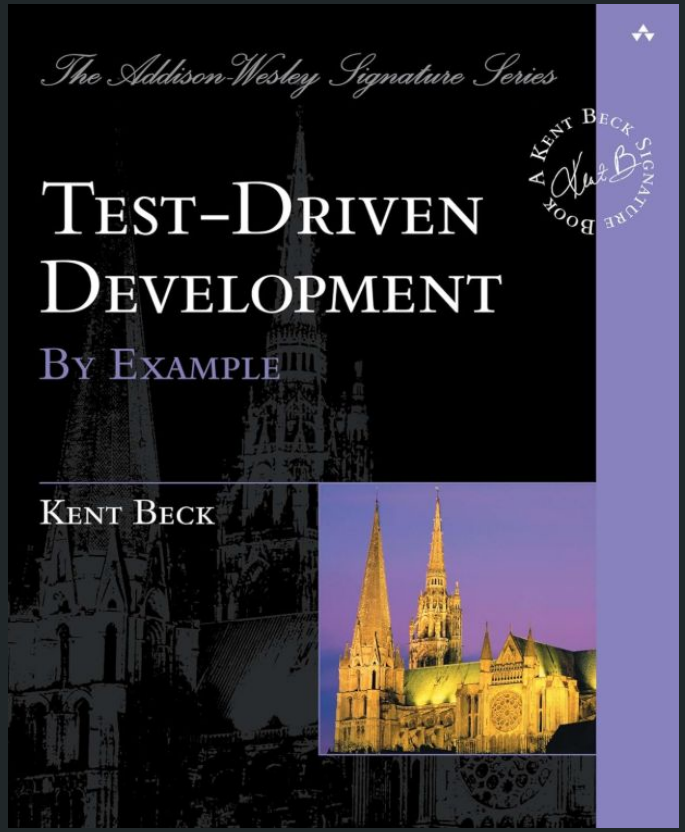


Retro time!

TDD

Test-Driven Development

Une recette de cuisine
“*eXtreme*”



“***Test-Driven Development***
is a way of managing fear
during programming”
Kent Beck

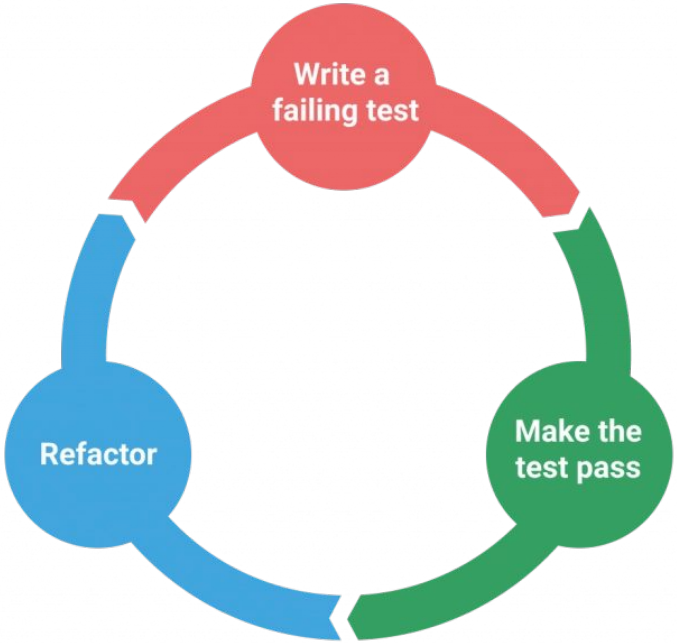
Les 3 lois de TDD

- “You are not allowed to write production code **until you have a failing test**”

- “You are not allowed to write any more of a test than **is sufficient to fail** (compilation failures are failures”

- “You are not allowed to any more production code than **is sufficient to pass the one failing test**”

Les 3 cycles de TDD - Red/Green/Blue



***Source**: TDD Manifesto - https://tddmanifesto.com/getting-started/*

Step Red - Write a failing test

- Etape **obligatoire**: on vérifie **absolument** que le test échoue avant de passer à l'étape suivante
- Le test définit un comportement qui n'existe pas encore dans notre application
- Un test pensé du **point de vue d'un utilisateur** (de notre code)
- Comment par un test nous force à écrire un code... **testable**

Step Green - Make it pass

- Faire passer notre test écrit lors de la step Red... **le plus vite possible**!

- C’est open-bar. Vous pouvez:
  - Dupliquer
  - Hardcoder des valeurs
  - Commenter du code
  - Ecrire un code ‘mauvais’

- On écrit le code **minimal** pour faire passer notre test; **minimal** ne veut pas dire **fonctionnel**

Step Blue - Refactor (Make it clean!)

- **La belle vie**. On est **sécurisé** et **serein**
- Step dans laquelle on veut rester le plus longtemps
- On nettoie les mauvaises choses que l'on a fait en step Green
- **On n'ajoute pas de nouveau comportement!**

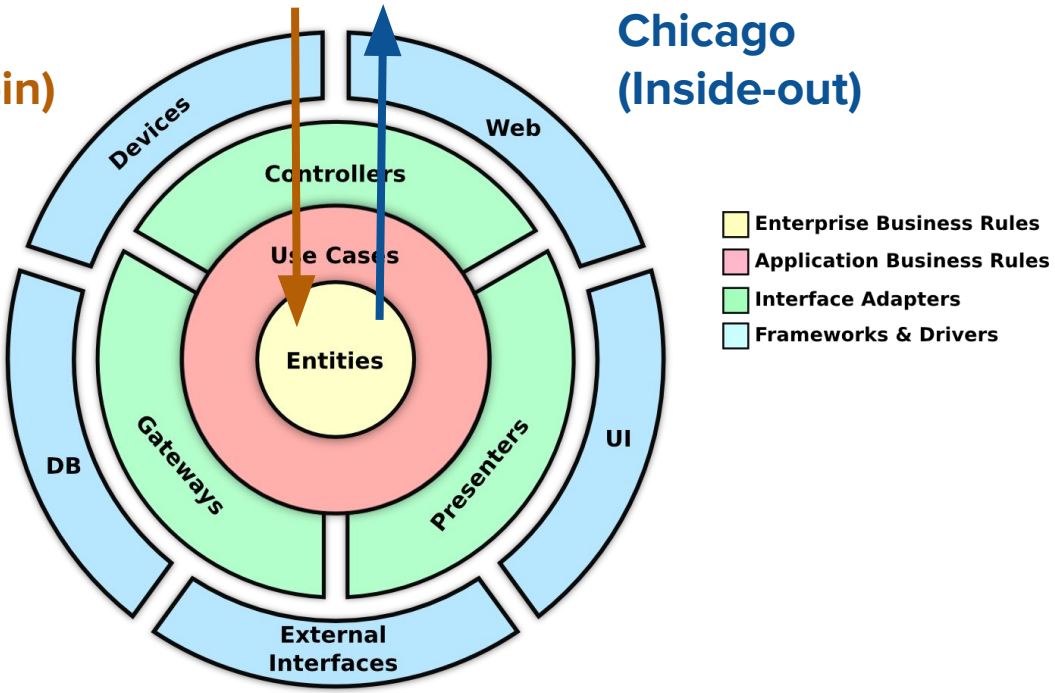
Les concepts clés

- Test-Driven Development **!=** écrire de**s** test**s** en premier (**Tests-First**)
- Les tests expriment l'API **idéale** avant que l'implémentation existe
- Les tests garantissent **un contrat** (what); pas une implémentation (how)
- Les tests nous poussent **naturellement** vers un bon design

Les écoles TDD: **Chicago** vs. **London**

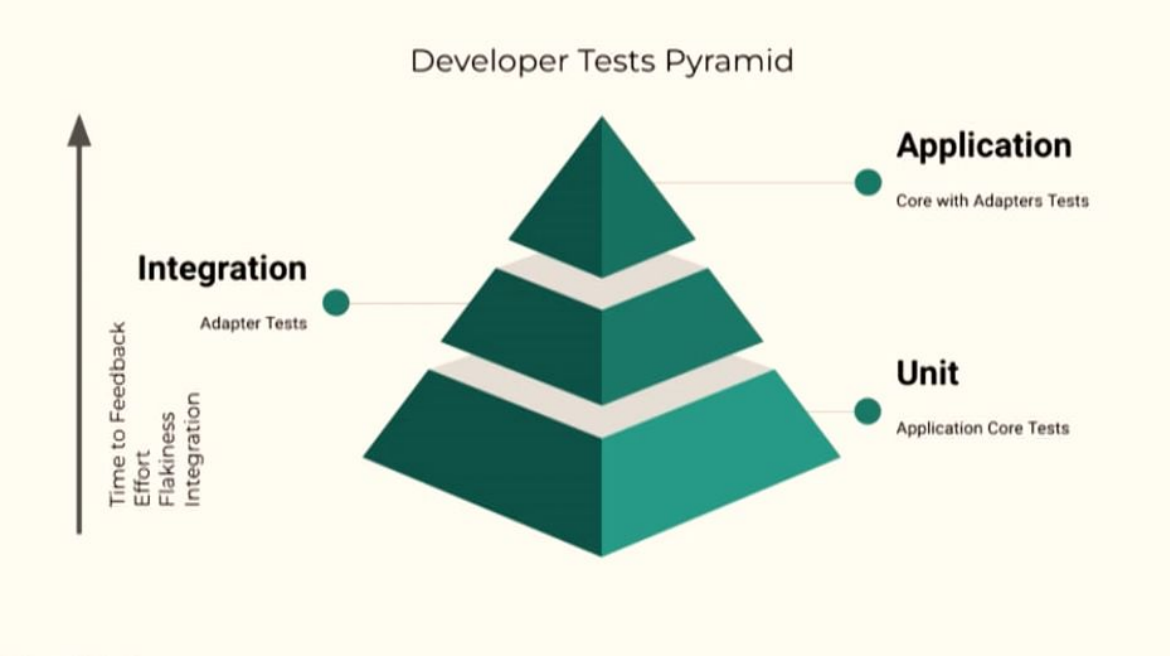
London
(Outside-in)

Chicago
(Inside-out)



Source: *https://pitchart.github.io/ddd-cqrs-mvc/#/*

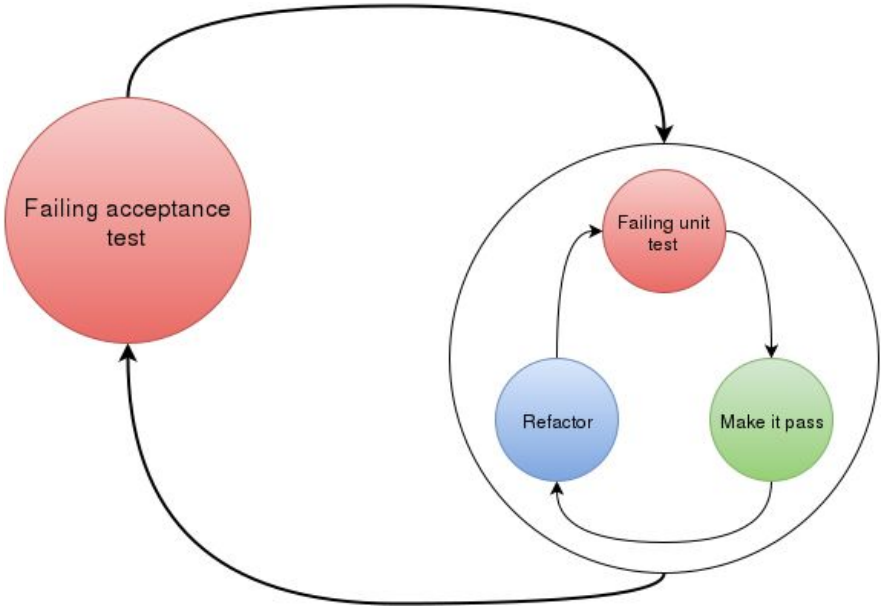
Uniquement pour des tests unitaires?



Source: *Guilherme Ferreira - The Grand Unified Theory of Clean Architecture and Test Pyramid*

https://www.youtube.com/watch?v=8BuCzpfp8Pk

Uniquement pour des tests unitaires?



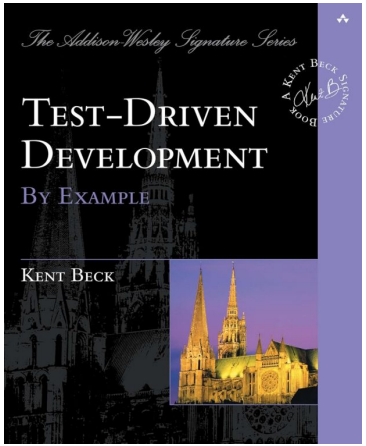
Source: *https://www.codurance.com/publications/2018/06/17/frontend-outside-in*

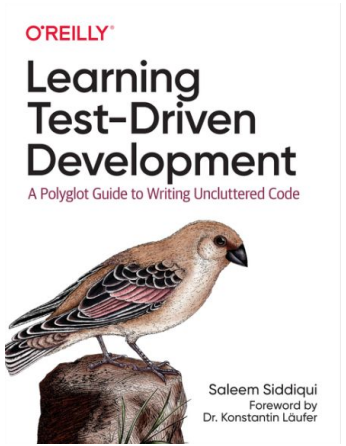
Aller plus loin avec TDD

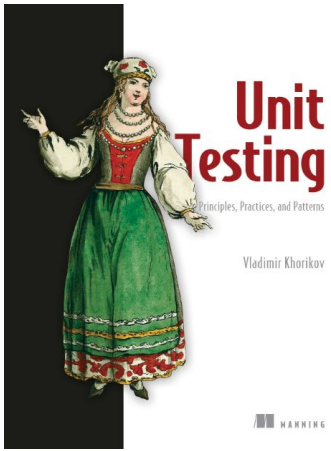
- Réduire encore les cycles (**baby steps**)... pour accélérer les feedback-loops
- Vert == Commit? (**Test && Commit**)...
- Commit == Release? (**Continuous Delivery**)
- Régression == Rollback? (**Test && Commit** || **Revert**)

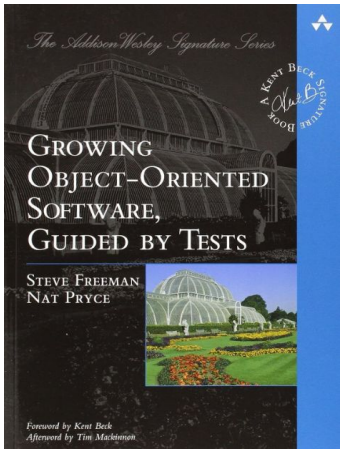
Resources

- https://tddmanifesto.com
- Xtrem-TDD Knowledge base: https://xtrem-tdd.netlify.app/
- Sandro Mancuso - “Outside-in TDD”: https://www.youtube.com/watch?v=XHnuMjah6ps
- Ray Osherove - “Refactoring and design techniques for TDD”: https://www.youtube.com/watch?v=QbNhpPQkCBs
- Sandro Mancuso: “Does TDD really lead to good design”: https://www.youtube.com/watch?v=KyFVA4Spcgg







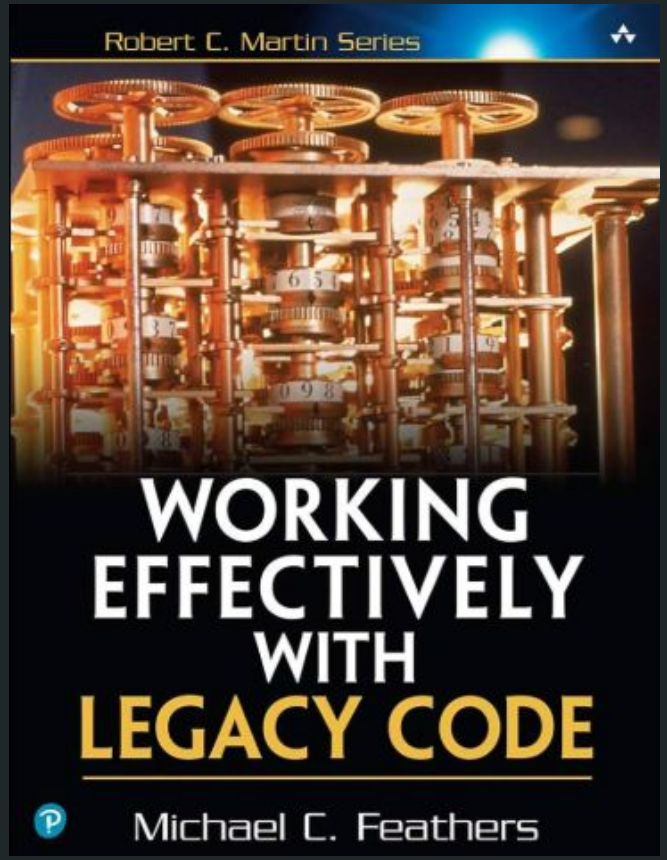


Retro time!

Code Legacy

C’est quoi un code “**legacy**”?

Un projet devient legacy dès qu’il fête son 3ème anniversaire…
Non?



Objectif N°1:
Réduction des risques

Changement sans test = régression… Enfin, **presque**

- **Chaque** changement **humain** est sujet à un risque

- Certains changements **automatisés** sont considérés comme **sûrs**:
  - Extract method
  - Rename
  - Introduce parameter
  - Inline variable
  - Move function

Reprendre le contrôle: **Golden Master / Approval Testing**

- Empêcher les régressions **sans comprendre le code**

- “Capture” du comportement à un instant **T**

Reprendre le contrôle: **Example Mapping**

- Faire **émerger** les règles métier
  - Exemples -> Règles -> Comportement + Questions

- Reconstituer l’intention du code

- Transformer les comportements en tests, sur base des exemples

- Les questions servent à reconstruire un échange avec le business

Reprendre le contrôle: **Boy Scout Rule**

- Ne jamais réécrire tout le legacy
- Améliorer **uniquement la partie touchée**
- Ajustement **continus**; pas de “big bang”

Cependant, un code legacy est rarement testable… **facilement**

Vous avez dit “**coupling**”?

Utiliser des “Seams” pour intégrer des tests

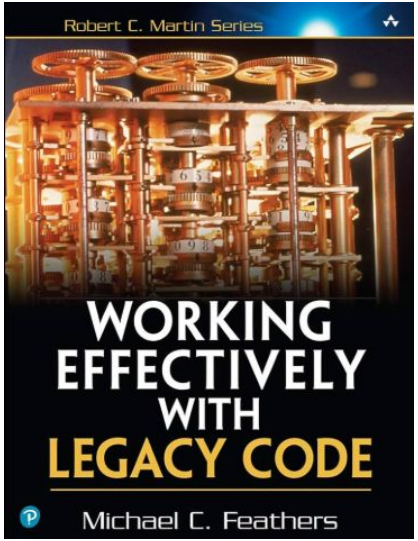
- “Pouvoir changer le comportement d’un dépendance **sans modifier le code métier**”

- Isole et remplace une dépendance externe, **malgré le couplage**

- Un outil pour reprendre le contrôle sur le code

Resources

- Sandro Mancuso - “Testing and Refactoring Legacy Code”: https://www.youtube.com/watch?v=LSqbXorkyfQ
- Emily Bache - “Approval Testing”: https://www.youtube.com/watch?v=0ZVKcFsEp-4



Retro time!

Load Testing

Pourquoi se reposer sur du Load Testing

- Le code fonctionne en local (ou dans un test)… mais pas toujours dans une utilisation “réelle”
- La charge réelle est difficilement prévisible
- Eviter le “*sur-provisioning*”
- Permet d’avoir des garanties sur ce que notre application peut supporter

Les différents types de Load Testing

Type	Objectif
Smoke Test / Health Test	Vérifier que cela répond
Load Test	Vérifier une charge réaliste
Stress Test	Vérifier une limite de capacité
Spike Test	Vérifier une montée en charge très rapide
Soak Test	Vérifier l’état du système après une charge “longue-durée”

Comment on définit une charge?

- **VU**s: Utilisateurs virtuels
- **RPS** / **TPS**: Le “*throughput*”
- **Latency**: Les percentile (P50, P90, P95)
- **Saturation**: Visualiser le point d’effondrement

Un Load Test doit diriger vers des optimisations

- Quel est le trafic **réel** ou **attendu**?
- Quelle est notre **capacité actuelle**?
- Où sont les **goulots d'étranglement**?
- Quelle charge à un impact sur l'expérience utilisateur?
- Quel seuil pour une alerte?

Disclaimer

- Un *load test* ne s'effectue **jamais en local**
- Toujours sur une infrastructure **déployée** (staging, preprod, etc)
- Environnement **similaire** de la production (**data incluses**)
- Test & Dev -> Build Pipeline (Test) -> Deploy -> Load Testing

Notre outil: **Grafana k6**

- k6: https://k6.io/

- Installation: https://grafana.com/docs/k6/latest/set-up/install-k6/

Interprétation des valeurs

```
THRESHOLDS

  http_req_duration{expected_response:true}
  ✗ 'p(90)<100' p(90)=957.03ms
  ✗ 'p(95)<200' p(95)=1.09s
  ✗ 'p(99)<500' p(99)=1.24s

  http_req_failed
  ✓ 'rate==0.00' rate=0.00%


TOTAL RESULTS

  checks_total.......: 14435   961.834203/s
  checks_succeeded...: 100.00% 14435 out of 14435
  checks_failed......: 0.00%   0 out of 14435

  ✓ OK response

  HTTP
  http_req_duration..............: avg=538.69ms min=20.14ms med=503.24ms max=1.45s p(90)=957.03ms p(95)=1.09s
    { expected_response:true }...: avg=538.69ms min=20.14ms med=503.24ms max=1.45s p(90)=957.03ms p(95)=1.09s
  http_req_failed................: 0.00%  0 out of 14435
  http_reqs......................: 14435  961.834203/s

  EXECUTION
  iteration_duration.............: avg=538.81ms min=20.18ms med=503.62ms max=1.45s p(90)=957.17ms p(95)=1.09s
  iterations.....................: 14435  961.834203/s
  vus............................: 12     min=12         max=995
  vus_max........................: 1000   min=1000       max=1000

  NETWORK
  data_received..................: 3.3 MB 220 kB/s
  data_sent......................: 1.1 MB 75 kB/s



unning (15.0s), 0000/1000 VUs, 14435 complete and 0 interrupted iterations
efault ✓ [======================================] 0000/1000 VUs  15s
```

Interprétation des valeurs

- ‘**med=503.24ms**’: Le temps médian, soit plus de 50% des requêtes prennent plus que 503,24ms.
- Les percentiles:
  - ‘**p(90)=957.03ms**’: 10% des requêtes prennent plus que 957.03ms.
  - ‘**p(95)=1.09s**’: 5% des requêtes prennent plus que 1,09s.
  - ‘**max=1.45s**’: La requête la plus lente.
- Throughput: 14435 requêtes exécutées, soit **961 requêtes par seconde**.
- ‘**med**’ vs. ‘**min**’
  - ‘**min**’ = 20ms
  - ‘**med**’ = 503.24ms

Retro time!

Fin du cours

Merci à tous!